

Practica 6 - Parametros

Algunos tips:

Parametro Real (argumento):

- Es el valor que se le pasa cuando se **llama/invoca** a la rutina

```
saludar("Gianca");  
// Gianca es el parametro real (argumento)
```

Parametro Formal:

- Es el nombre que se usa en la definicion de una funcion o metodo
- Actua como variable local dentro de esa funcion

```
public void saludar (String unNombre)  
{  
    System.out.println("Hola " + unNombre);  
}  
// unNombre es el parametro formal.
```

Ligadura de parametros:

- Posicional: Se corresponden con la posicion que ocupan en la lista de parametros.
- Palabra clave o nombre: Se corresponden por el nombre, por lo tanto pueden estar colocados de manera desordenada en la lista de parametros.

En ADA pueden mezclarse ambos metodos de ligadura de parametros.

Ademas, en C++ y ADA, los parametros formales pueden tener valores por defecto, con lo cual aveces no es necesario listarlos todos en la invocacion.

Parametros IN - OUT - IN/OUT

- Parametros IN: El parametro formal recibe el dato desde el parametro real
- Parametros OUT: El parametro formal envia el dato al parametro real
- Parametros IN/OUT: El parametro formal recibe el dato del parametro real y el parametro formal le envia el dato al parametro real.

Ejercicio 1:

Explique brevemente los siguientes conceptos

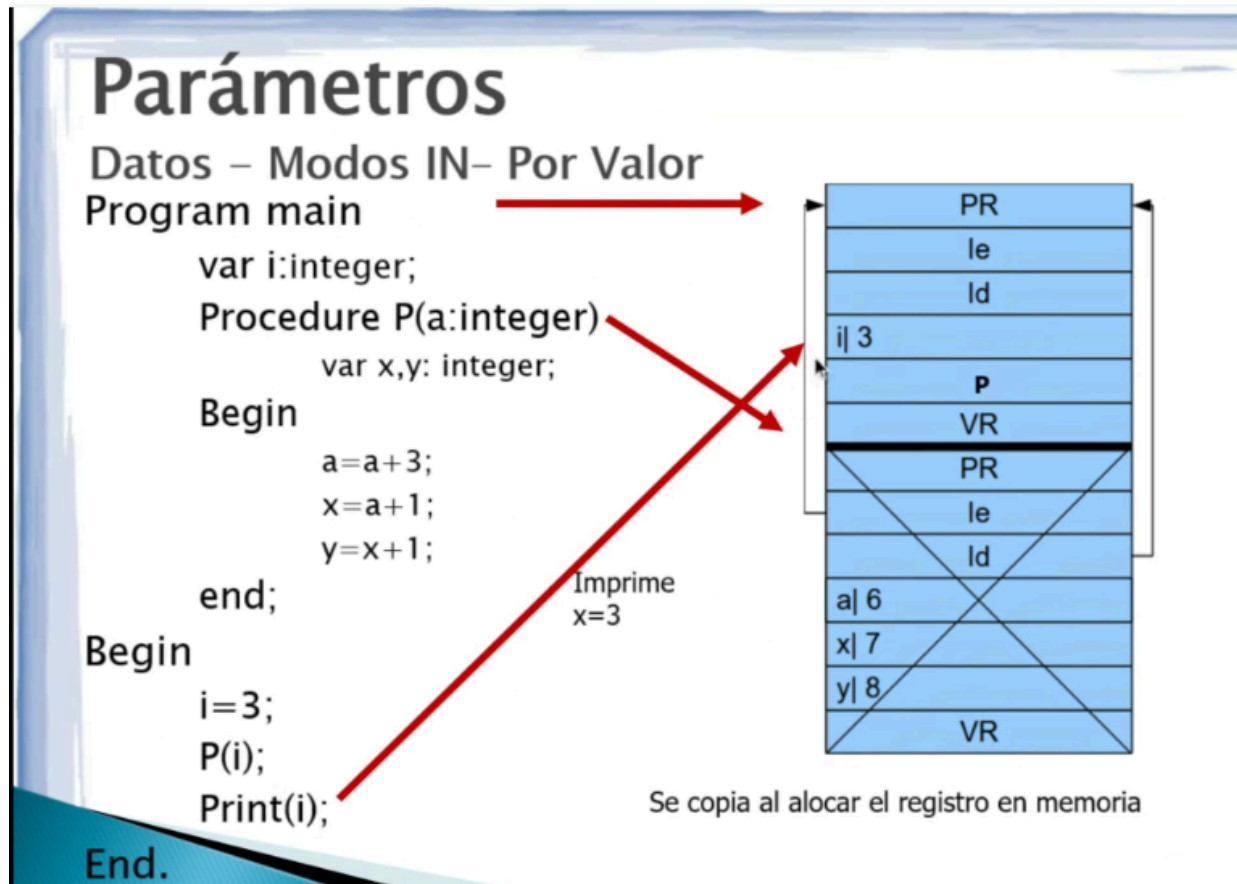
- **Parámetro:** Es una variable que se utiliza para enviar información a una rutina. Sirven para comunicar datos entre rutinas. Los parámetros son una forma de comunicación entre distintas partes del programa.
- **Parámetro real:** El parámetro real es el valor que se le pasa a la rutina/subprograma al momento de invocarlo. Se lo puede llamar parámetro real o argumento.
- **Parámetro formal:** El parámetro formal es una variable utilizada para recibir valores de entrada en una rutina, subrutina. Los parámetros formales están colocados en la parte de la **declaración** de la rutina. Actúan como variables locales dentro de ese subprograma.
- **Ligadura posicional:** Se corresponden con la posición en la lista de parámetros. Van en el mismo orden.
- **Ligadura por palabra clave o nombre:** Se corresponden con el nombre, por lo tanto pueden estar colocados en distinto orden.

Ejercicio 2:

Modo IN : El parámetro formal recibe el dato desde el parámetro real. La conexión es al inicio, se copia y corta la vinculación.

- **Modo IN , por valor:** El valor del parámetro real se usa para inicializar el correspondiente parámetro formal al invocar la unidad. Se transfiere el dato real y se copia.

- Ejemplo :

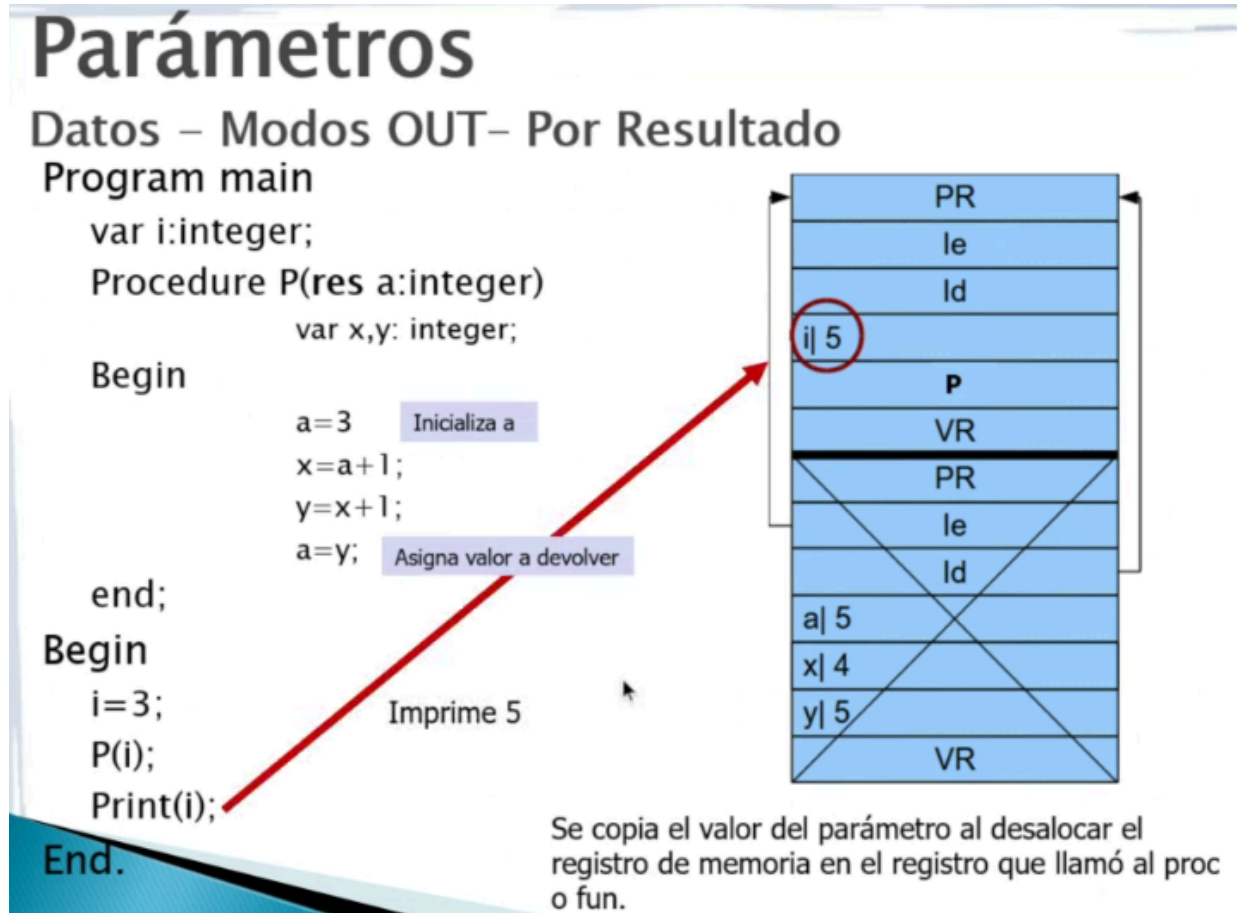


- Modo IN, por valor constante: Se envia un valor, pero la rutina receptora no puede modificarlo. Es decir, queda con un valor fijo que no puede cambiar. (dependera del lenguaje si se realiza o no la copia)
- La implementacion debe verificar que el parametro real no sea modificado.

Modo OUT: Se invoca la rutina y cuando esta termina devuelve el parametro formal al parametro real. La conexion es al final.

- Modo OUT, por resultado: El valor del parametro formal de la rutina se copia al parametro real al terminar de ejecutarse la unidad que fue llamada. El parametro formal es una variable sin valor inicial por que no recibe nada. **Debemos inicializar de alguna forma la variable** si el lenguaje no lo hace por defecto, sino puede traer problemas.

- Ejemplo :



- Modo OUT por resultado de funciones: Es el resultado de lo que devuelve la funcion. Existen distintas formas para retornar un valor dentro de una funcion
- Ejemplo:

<u>En C</u>	<u>En Pascal</u>
int f1(int m);	Function F1(m:integer):integer;
{....	begin
return(m)	F1:=m + 5;
}	end;

Modo IN/OUT: El parametro formal recibe el dato del parametro real y el parametro formal le envia el dato al parametro real al finalizar la rutina. La conexion es al inicio y al final.

- Modo IN/OUT por valor_resultado:El parametro formal es una variable local que recibe una copia a la entrada del contenido del parametro real y a la salida el parametro real recibe una copia de lo que tiene el parametro formal. Una vez que termina de ejecutar el parametro formal le devuelve un valor al parametro real y lo copia.

- Ejemplo:

Parámetros

Modos IN/OUT- Por Valor/Resultado

Program main

```

var i:integer;
Procedure P(in-out a:integer)
    var x,y: integer;

    Begin
        a=4;
        x=a+1;
        y=x+1;
        a=y;
    end;
Begin
    i=3;
    P(i);
    Print(i);
End.

```

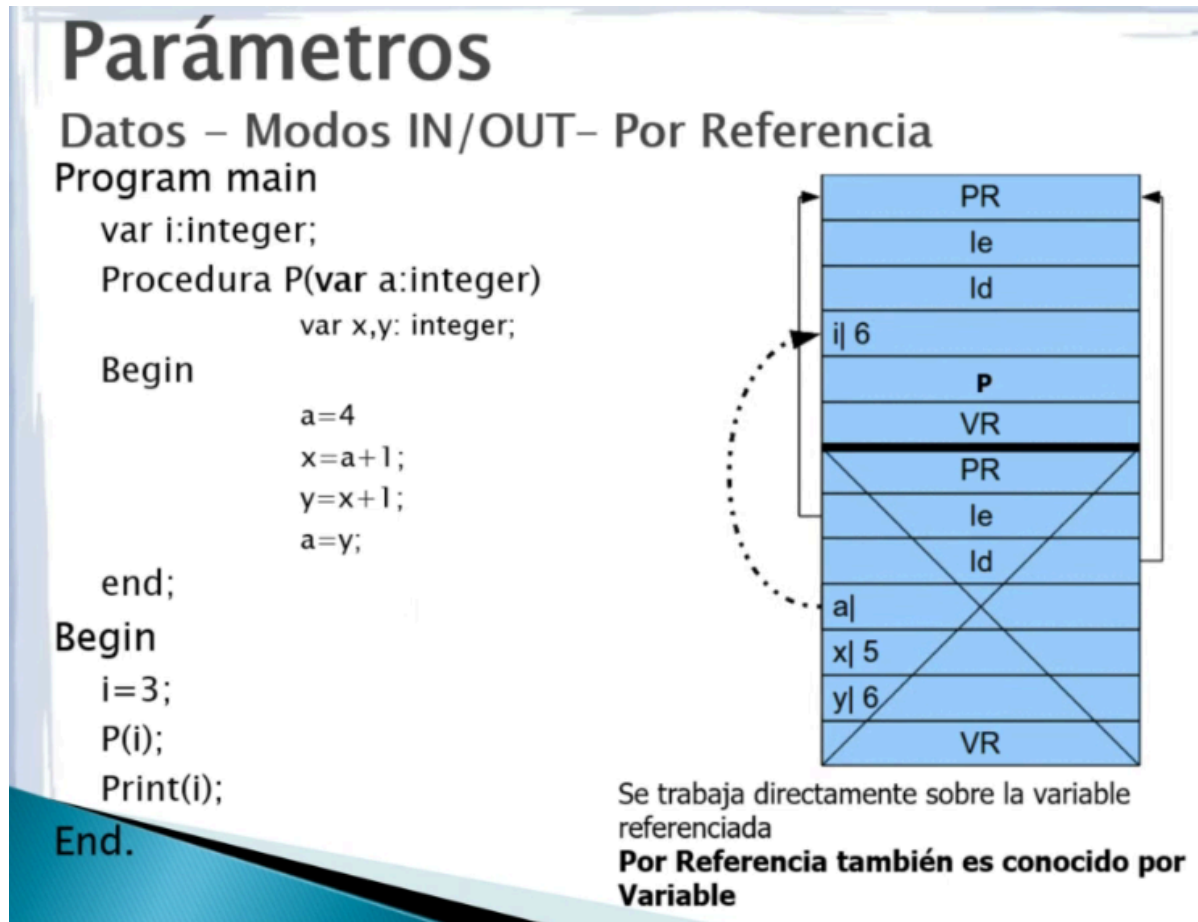
Se copia al alocar el registro y se modifica el parámetro formal al finalizar la ejecución de la rutina.

- Modo IN/OUT, por referencia: También llamada por "variable". NO se le hace una copia, si no que se le pasa la referencia/dirección (L-Valor) del parámetro real al parámetro formal.

La conexión es al inicio y permanece hasta el final . La conexión es permanente.

Cualquier cambio que se realice en el parámetro formal dentro del subprograma, quedara registrado en el parámetro real. El cambio sera automatico.

- Ejemplo:



- Modo IN/OUT por nombre: El parametro formal es sustituido "textualmente" por una expresion del parametro real.

El pasaje por nombre toma *textualmente* la expresión que le mandamos, como si la reemplazara en el código del procedimiento cada vez que se usa.

Esto quiere decir que **si pasás $a + 1$** , se evaluará **$a + 1$ cada vez que se use el parámetro dentro del procedimiento**, no una sola vez.

- Ejemplo:

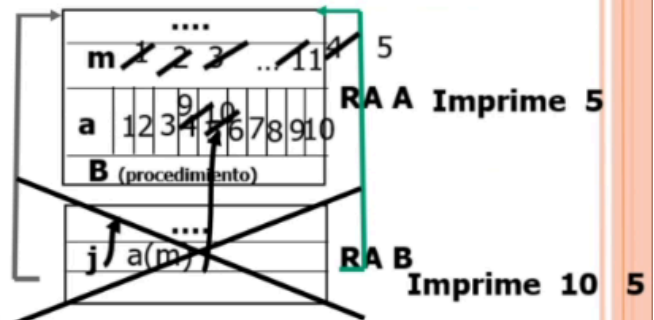
Parámetros

Datos – Modos IN/OUT–
Por Nombre

```

Procedure A ();
    var m:integer;
    a: array[1..10] of integer;
Procedure B (nombre j:integer);
    begin
        j:=j+5;
        m:=m+1;
        j:= j+ m;  write (j,m);
    end;
begin
    for m=1 to 10 begin  a[m]=m;  end;
    m:=4;  B(a[m]);
    write (m);
end;

```



Es fundamental preguntarse ¿qué significa "a(m)" en el registro de activación del "llamante". Este es el secreto del parámetro por nombre. Calcula la dirección durante la ejecución

54

Ejercicio 3

a) complete el siguiente cuadro segun lo correspondiente a cada lenguaje:

Tipo de pasaje de parametros	Lenguaje
Por copia Modo IN, Por Resultado modo OUT IN/OUT. Para los tipos de datos primitivos indica que es por valor-resultado Para los tipos no primitivos y datos compuestos (arreglos,registros) se hace por referencia	ADA
Por valor Si se necesita por referencia se usan punteros. Permite pasaje por valro constante, agregandole const	C
Por valor Pero si se pasa un objeto mutable, no se hace una copia, si no que se trabaja sobre el.	Ruby
Solo copia del valor.	Java

Tipo de pasaje de parametros	Lenguaje
Envia objetos que pueden ser mutables o inmutables. Si es inmutable actuara como por valor si es mutable ejemplo listas, no se hace una copia, si no que se trabaja sobre el.	Python

b) Ada es mas seguro que pascal con respecto al pasaje de parametros por que obliga a especificar la intencion del uso de parametros. (in, out, in/out) y verifica su uso correcto en tiempo de compilacion.

c) El pasaje de parametros in/out depende del tipo de dato.

Para los tipos de datos escalares: Como Integer, Boolean, Float, el compilador puede usar pasaje por valor o por copia temporal.

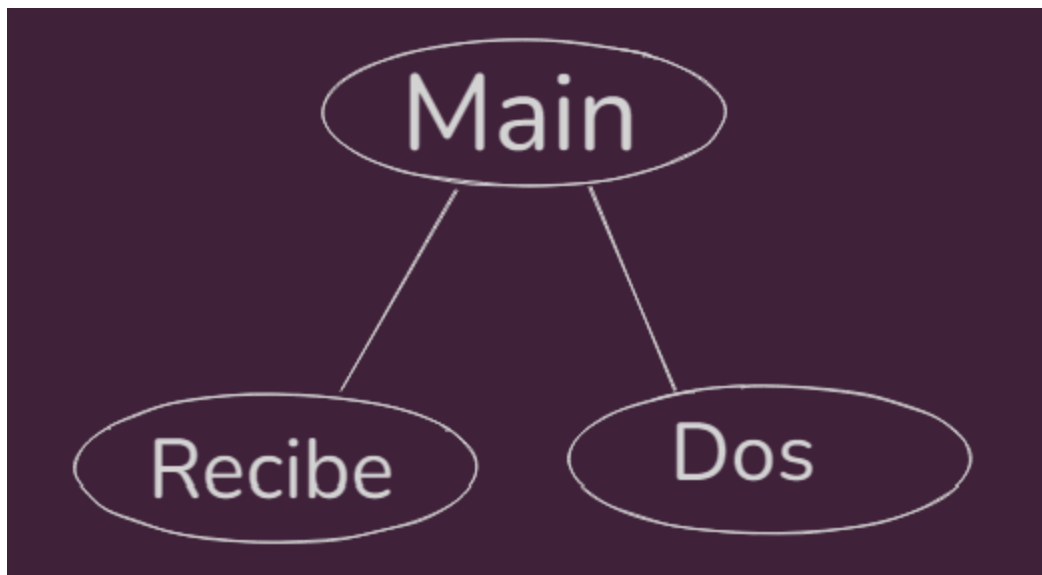
Para los tipos de datos compuestos como arrays, records, en general se pasan por referencia ya que son mas grandes.

Para los tipos acces (punteros), son referencias.

Ejercicio 4:

A)

arbol de derivacion:



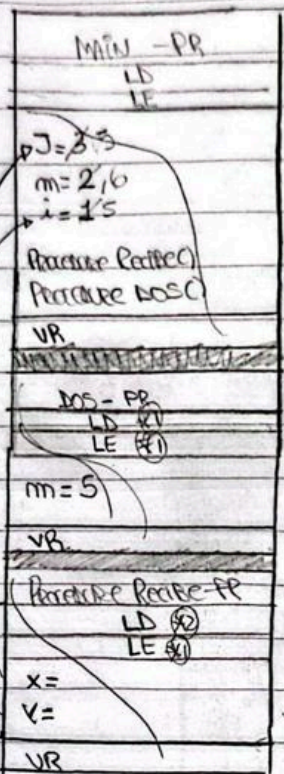
B)

i) Por referencia:

Ejercicio 4:

Recursión

ANALISIS ESTÁTICO:



$$m = 2 + 1 + J = 6$$

$$i = i + x + 3 = i = 1 + 1 + 3 = 5$$

$$J = m - 1 = J = 6 - 1 = 5$$

write(i, J, m)
Imprime: 5, 5, 6

write(i, J, m)
Imprime: 5, 5, 5

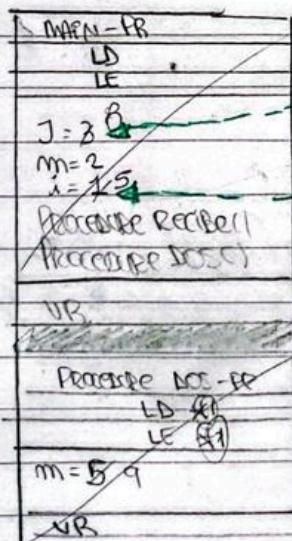
$$m = m + 1 + y =$$

$$x = i + x + J$$

$$y = m - 1$$

write(x, y, 1, 2, m)
Imprime: 5, 5, 5, 5, 6

Recursión - CÁLCULO DINÁMICO



write(i, J, m)
Imprime: 5, 8, 2

$$x = i + x + J$$

$$x = 1 + 1 + 3$$

$$y = m - 1$$

$$y = 9 - 1$$

write(x, y, 1, 1, J, m)
Imprime: 5, 8, 5, 8, 9

$$m = m + 1 + y$$

$$m = 5 + 1 + 3 = 9$$

write(i, J, m)
Imprime: 5, 8, 9

Papel de fibra de caña de azúcar.

CS

Escaneado con CamScanner

ii) Por valor:

II VALOR - ADJUNTA ESTÁTICA

x1	MAIN-PR
	LD
	LE
	j=3
	m=26
	i=1
	Procedure Recur()
	Procedure Loco()
	VR
x2	Procedure Rec-PR
	LD 40
	LE 41
	m=5
	VR
x3	Procedure Rec-PR
	LD 42
	LE 43
	x=1/5
	y=3/5
	VR

WRITE(i,j,m)
IMPRIME(1,3,6)

WRITE(i,j,m)
IMPRIME(1,3,5)

m = m + 1 + y
m = 2 + 1 + 3 = 6
x = x + x + 3
x = 1 + 1 + 3 = 5

y = m - 1
y = 6 - 1 = 5

WRITE(x,y,i,j,m)
IMPRIME(5,5,1,3,6)

VALOR , ADJUNTA DINÂMICA

x1	MAIN-PR
	LD
	LE
	j=3
	m=2
	i=1
	Procedure Recur()
	Procedure Recur()
	VR
x2	Procedure Rec-PR
	LD 40
	LE 41
	m=5/9
	VR
x3	Procedure Rec-PR
	LD 42
	LE 43
	x=1/5
	y=3/8
	VR

WRITE(i,j,m)
IMPRIME(1,3,2)

WRITE(i,j,m)
IMPRIME(1,3,9)

m = m + 1 + y
m = 5 + 1 + 3 = 9
x = x + x + 3
x = 1 + 1 + 3 = 5
y = m - 1
y = 9 - 1 = 8

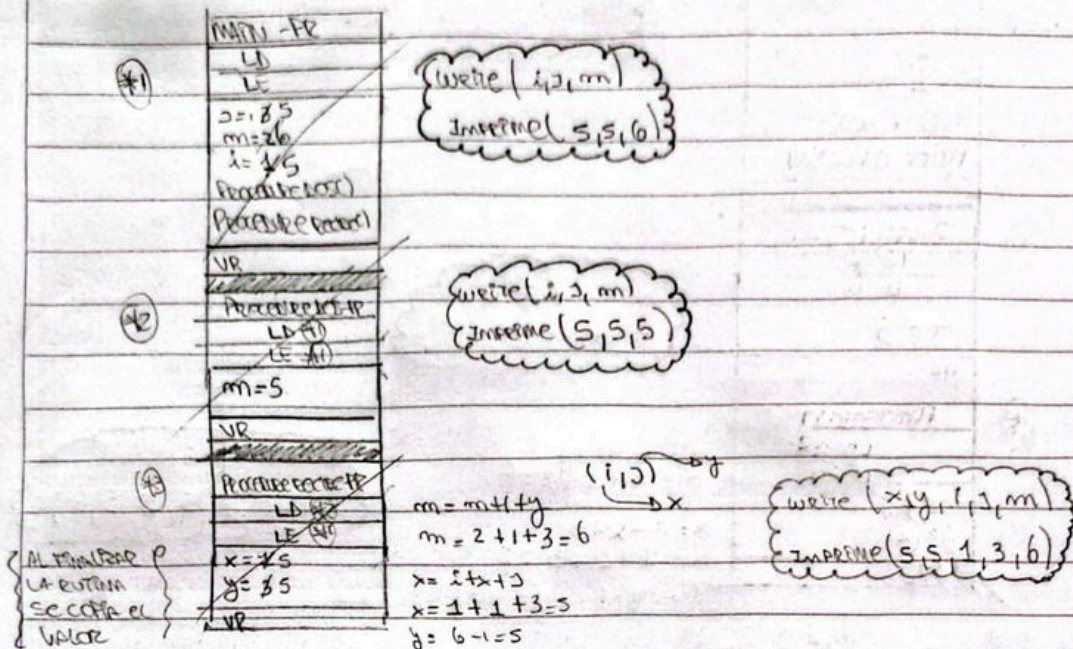
WRITE(x,y,i,j,m)
IMPRIME(5,8,1,3,9)



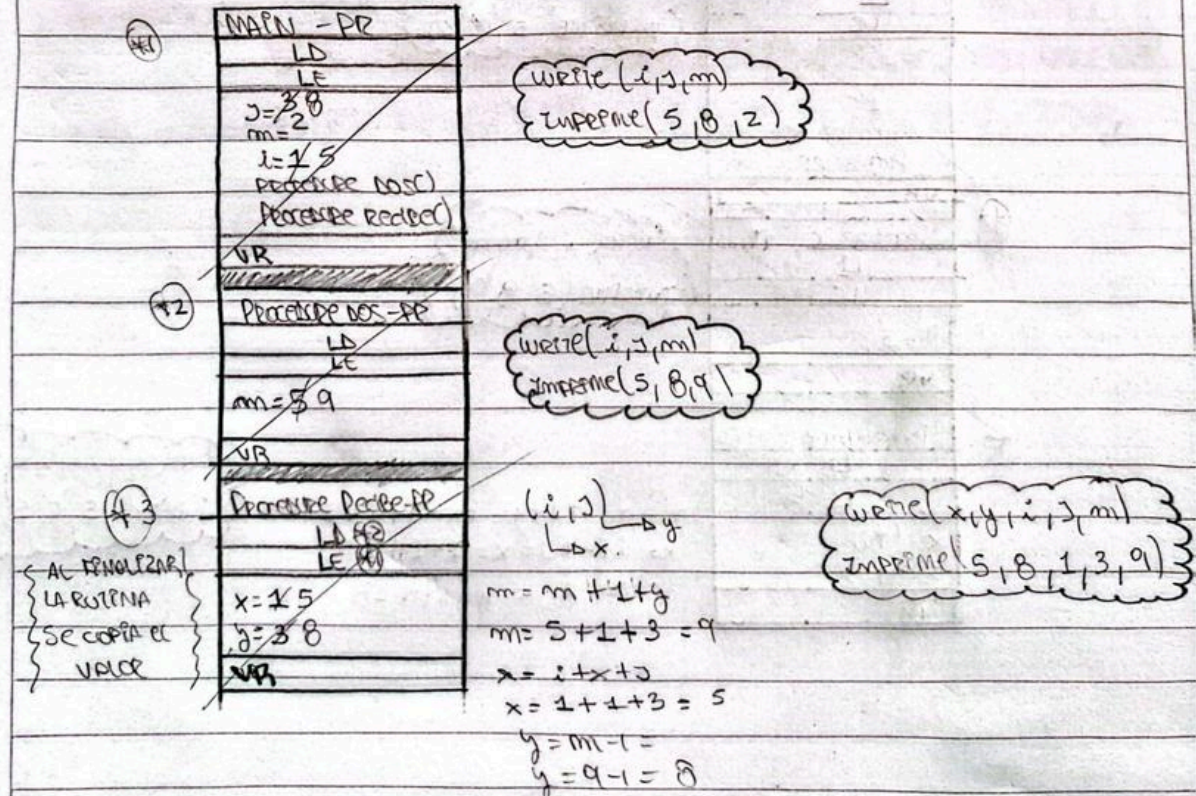
iii) Por valor-resultado:

E. PARAMETRO TAMAN SECCION AL PRIMER PUNTO

III VALOR-RESULTADO, CADA UN ESTADO



VALOR - RESULTADO, CADA UN DINAMICA



iv) Por nombre:

IV - NOMEZE - CADERNATA

(*)	main - PR
	LD
	LE
	$j = 3/5$
	$m = 26$
	$i = 1/5$
	Procedure DOS
	Procedure RECEBER
	VR
(*)	Procedure DOS
	LD
	LE
	$m = 5$
	VR
(*)	Procedure RECEBER
	LD
	LE
	$x = (1/1)$
	$y = (1/3)$
	VR

write (i, j, m)
imprime (5, 5, 6)

write (i, j, m)
imprime (5, 5, 5)

$$m = m + 1 + j$$

$$m = 2 + 1 + 3 = 6$$

$$x = i + x + j =$$

$$x = 1 + 1 + 3 = 5$$

$$y = m - 1 = y = 6 - 1$$

write (x, y, i, j, m)
imprime (5, 5, 5, 5, 6)

NOMEZE - CADERNATA

(*)	main - PR
	LD
	LE
	$j = 3/8$
	$m = 2$
	$i = 1/5$
	Procedure DOS
	Procedure RECEBER
	VR
(*)	Procedure DOS
	LD
	LE
	$m = 5/9$
	VR
(*)	Procedure RECEBER
	LD
	LE
	$x = (1/1)$
	$y = (1/3)$
	VR

write (i, j, m)
imprime (5, 8, 2)

write (i, j, m)
imprime (5, 8, 9)

$$m = m + 1 + j = m = 5 + 1 + 3 = 9$$

$$x = i + x + j = x = 1 + 1 + 3 = 5$$

$$y = m - 1 = y = 9 - 1 = 8$$

write (x, y, i, j, m)
imprime (5, 8, 5, 8, 9)

v) Por resultado:

NOTA: POR RESULTADO: El valor del parámetro formal se copia al parámetro real.
Al terminar de ejecutarse la unidad que fue llamada
debemos inicializar de alguna forma la variable.

✓ Por resultado

Se produce un error en el procedimiento recibir (x :integer; y :integer)
Este procedimiento recibe dos parámetros por resultado
y puede acceder al valor de los parámetros para poder inicializarlos.
Esto está mal; cuando se usa el modo OUT por resultado,
la rutina está obligada a inicializar la variable con un valor
que no dependa de su variable (su contenido).

Las líneas que generan error son:

$m := m + 1 + y$; { ERROR, se está leyendo
el parámetro y , que NO
está inicializado }

$x := i + x + j$; { ERROR, se está leyendo
el parámetro x , que NO
está inicializado }

CONCLUSIÓN: Cuando trabajamos con parámetros por resultado
la rutina está obligada a inicializar los valores
antes de usarlos, sino lo hace, se genera error.

El unico caso que no se pudo realizar fue el caso de pasaje de parametro modo out por resultado. Por que el procedimiento Recibe, tiene dos parametros ("x" , "y"), pero cuando se utiliza la tecnica de pasaje por resultado, la rutina está obligada a asignarle un valor al parametro antes de utilizarlo. Este valor que se le asigna debe ser independiente de dicho parametro.

Y como dentro del procedimiento Recibe, se quiere inicializar los parametros con valores que aun no estan definidos, se termina generando error.

```
begin
  m := m + 1 + y;    // 🚨 ERROR: estás leyendo y
  x := i + x + j;    // 🚨 ERROR: estás leyendo x
  y := m - 1;        // ✅ OK: estás asignando y
  write(x, y, i, j, m);
end;
```

Conclusion: Se generan los dos errores en pasaje por resultado por que:

- Su contenido es **indefinido** al comienzo del procedimiento.
- Se espera que el procedimiento los **escriba** antes de usarlos.

D)

No, no da el mismo resultado si se utiliza cadena dinamica. El resultado varia dependiendo del tipo de cadena que utilice el lenguaje. Tenemos como ejemplo la resolucion del pasaje de parametros por valor, donde siempre que se hace un write, los valores varian dependiendo del tipo de cadena que se esté siguiendo.

Ejercicio 5:

A)

procedure rutina (referencia iteracion, referencia vector, referencia vir)

B)

procedure rutina (referencia iteracion, valor vector, referencia vir)

C)

procedure rutina (valor iteracion, valor vector, valor vir)

Ejercicio 6:

Un valor entero: Un único valor se comporta exactamente igual que el pasaje por referencia


```

procedure MostrarDosVeces(x: integer);
begin
    writeln(x);
    writeln(x);
end;

MostrarDosVeces(5);
// El resultado esperado es:
// 5
// 5

```

Una constante: Si es una constante es equivalente a por valor

```

const a = 10;
procedure MostrarDosVeces(x: integer);
begin
    writeln(x);
    writeln(x);
end;

MostrarDosVeces(5);
// El resultado esperado es:
// 10
// 10

```

Un elemento de un arreglo: Si es un elemento de un arreglo puede cambiar el suscripto entre las distintas referencias

```

var
    arr: array[1..3] of integer = (1, 2, 3);
    i: integer = 1;
procedure Incrementar(x: integer);
begin
    x := x + 1;
end;

begin
    Incrementar(arr[i]);

//● En Pascal esto pasa por valor, así que `arr[i]` no se modifica.
//Pero si se pasara por nombre, lo que ocurre es que:

x := x + 1;

```

```
// Se convierte en
```

```
arr[i] := arr[i] + 1;
```

Entonces el arreglo sí se modifica.

Resultado si fuera por nombre:

`arr[1]` cambia de `1` a `2`.

Una expresión: Si es una expresión se evalúa cada vez

```
var
```

```
  a: integer = 3;
```

```
procedure Mostrar(x: integer);
```

```
begin
```

```
  writeln(x);
```

```
end;
```

```
begin
```

```
  Mostrar(a + 1);
```

```
//  Si fuera por valor, imprime `4`
```

```
//  Si fuera por nombre, cada vez que se usa `x`, se evalúa `a + 1`.
```

Ejemplo :

```
procedure Mostrar(x: integer);
```

```
begin
```

```
  writeln(x);
```

```
  a := a + 1;
```

```
  writeln(x);
```

```
end;
```

```
// Con pasaje por nombre el resultado seria:
```

```
4
```

```
5
```

Ejercicio 7:

7) ADON ESTADIA Y DINAMICA

PROCEDURE UNO - PR
LD
LE
 $y = 1, 2, 3, 4, 5, 6, 7, 8, 9, 10$
 $z = 2, 3$
 $R1[1] = 2$
 $R1[2] = 2$
 $R1[3] = 2, 3, 5$
 $R1[4] = 2$
 $R1[5] = 2$
 $R1[6] = 2$
 $R2[1] = 1$
 $R2[2] = 1, 2, 3, 4$
 $R2[3] = 1, 2, 3, 5$
 $R2[4] = 1$
 $R2[5] = 1$

PROCEDURE DOS

VR

PROCEDURE DOS - PR
LD
LE
 $X = R1(y + R2(y))$ {NOMBRES}
 $T = R2(z)$ {NOMBRES}
 $y = 1/3$ {valor real}
 $z = 1/3$

PROCEDURE DOS - PR

LD R2

LE

$T1 := R2(z)$ {NOMBRES}

PROCEDURE TRES

VR

PROCEDURE TRES - PR
LD R3
LE

VR

PRIMER FOR PARA IMPRIMIR

ITERACION IMPRIME

① → 2
 ② → 2
 ③ → 5
 ④ → 2
 ⑤ → 2
 ⑥ → 2

SEGUNDO FOR PARA IMPRIMIR

ITERACION IMPRIME

① → 1
 ② → 4
 ③ → 5
 ④ → 1
 ⑤ → 1

$$R1(y + R2(y)) = R1(2 + R2(2)) = R1(2 + 1) = R1(3) = 2$$

$$X := X + 1$$

$$R1(y + R2(y)) := R1(y + R2(y)) + 1$$

$$R1(2 + R2(2)) := R1(2 + R2(2)) + 1$$

$$R1(2 + 1) := R1(2 + 1) + 1$$

$$R1(3) := R1(3) + 1 = 3$$

$$T := T + 1$$

$$R2(z) := R2(z) + 1$$

$$R2(2) := 1 + 1 = 2$$

$$X := X + 2$$

$$R1(y + R2(y)) := R1(y + R2(y)) + 2$$

$$R1(2 + R2(2)) := R1(2 + R2(2)) + 2$$

$$R1(2 + 1) := R1(2 + 1) + 2$$

$$R1(3) := R1(3) + 2 = 5$$

$$T$$

$$T1 := T1 + 1$$

$$R2(z) := R2(z) + 1$$

$$R2(2) := R2(2) + 1 = 3$$

$$T := T + 1$$

$$R2(z) := R2(z) + 1 = 4$$

$$T1 := T1 + 2$$

$$R2(z) := R2(z) + 2$$

$$R2(3) := R2(3) + 2$$

$$R2(3) := 3$$

$$T := T + 2$$

$$R2(3) := R2(3) + 2$$

$$y := y + 1 = 3$$

$$z := z + 1 = 3$$



Ejercicio 8:

a)

SHALLOW (parecido a buscar una variable por cadena dinámica): El ambiente de referencia, es el del subprograma que tiene el parámetro formal subprograma.

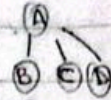
DEEP (parecido a buscar una variable por cadena estática): El ambiente es el del subprograma dónde está declarado el subprograma usado como parámetro real. Se utiliza en los lenguajes con alcance estático y estructura de bloque.

b)

Q. b) Given execution of program

Stack

↓ ADDRESS IN ADDRESS
BASE REGISTER



(1)

Program A - PR
LD
LE
x = 0
y = "a"
Procedure B
Procedure C
Function D

write(x,y) → inline(0, "a")

(2)

Procedure B - PR
LD 0
LE 0
H = 1
VR

write(y) → inline "a"
write(y) → inline "a"

(3)

Procedure C - PR
LD 0
LE 0
x = 3
y = "b" 5
VR 6

x = S(x,y) * write(x,y) → inline(6, "b")

(4)

Function S - PR
LD 0
LE 0
J = 8
K = "b"

J := J + x
K := y
J = 3 + 0 = 3
K = y → K = "b"
inline "b"

DEEP

(1)

Program A - PR
LD 0
LE 0
x = 0
y = "a"
Procedure B
Procedure C
Function D

write(x,y) → inline(0, "a")

(2)

Procedure B - PR
LD 0
LE 0
H = 1
VR

for write
write(y) → inline "a"
H = 1 + x = H = 1 + 0 = 1
write(y) → inline "a"

(3)

Procedure C - PR
LD 0
LE 0
x = 3
y = "b" 3
VR 3

x = S(x,y) write(x,y) inline(3, "b")

(4)

Function D - PR
LD 0
LE 0
J = 3
K = "a"

J := J + x
J = 3 + 0 = 3
inline "a"

Ejercicio 9:

PD: Gracias agus este punto te lo robe :)

a)

Nombre: El parámetro formal es sustituido textualmente por una expresión del parámetro real más un puntero al entorno del parámetro real. (se maneja una estructura aparte que resuelve esto). Se establece la ligadura entre parámetro formal y parámetro real en el momento de la invocación, pero la "ligadura de valor" se difiere hasta el momento en que se lo utiliza (la dirección se resuelve en ejecución). Distinto a por referencia. Es decir, no apunta a una dirección fija, puede ir cambiando (pero el nombre tiene que si ser el mismo)

Referencia: El parámetro formal será una variable local que contiene la dirección al parámetro real de la unidad llamadora que estará entonces en un ambiente no local. Cualquier cambio que se realice en el parámetro formal dentro del cuerpo del subprograma quedara registrado en el parámetro real.

Valor-Resultado: El parámetro formal es una variable local que recibe una copia (a la entrada) del contenido del parámetro real y el parámetro real (a la salida) recibe una copia de lo que tiene el parámetro formal. Básicamente lo que hace la rutina lo copio en la variable. Cada referencia al parámetro formal es una referencia local.

a) nombre

$$x=1$$

m

$$a(x) = a(x) + x + a(3)$$

$$a(1) = a(1) + 1 + a(3) = 3$$

$$x=2 \quad 1 + 1 + 1$$

$$a(3) = a(3) - 1 = 0$$

m

$$a(x) = a(x) + 1$$

$$a(2) = a(2) + 1$$

resultado

$$x=1$$

m

$$a(3) = a(3) + 1 + a(3) = 3$$

$$a(3) = a(3) - 1 = 2$$

m

$$a(3) = a(3) + 1$$

valor resultado

$$x=1$$

$$m = m + x + a(3) = 3$$

$$m = m + 1 = 4$$

	3	2	0				
a	x	x	x	1	1		

imprime 3, 2, 0, 1, 1

			3				
			2				
a	1	1	x	1	1		

imprime 1, 1, 3, 1, 1

			4				
			3				
a	1	1	x	1	1		

imprime 1, 1, 4, 1, 1

b)

b) nombre

a

1	1	3	1	1
---	---	---	---	---

m
 $a(3) = a(3) + 1 + a(3) = 3$ imprime 1, 1, 3, 1, 1
 $a(3) = a(3) - 1$
 m
 $a(3) = a(3) + 1$

referencia

a

1	1	3	1	1
---	---	---	---	---

$m =$
 $a(3) = a(3) + 1 + a(3) = 3$ imprime 1, 1, 3, 1, 1
 $a(3) = a(3) - 1$
 m
 $a(3) = a(3) + 1$

valor resultado

a

1	1	4	1	1
---	---	---	---	---

$m =$
 $m = 1 + 1 + 1 = 3$
 $m = m + 1 = 4$
 m
 3

FECHA

Ejercicio 10:

a)

El **pasaje por valor-resultado** combina **pasaje por valor** (al principio del procedimiento) y **pasaje por referencia** (cuando el valor de la variable es actualizado dentro del procedimiento). Es como un **comportamiento mixto**.

En **pasaje por valor-resultado**, el valor **inicial** de la variable se pasa por valor (lo que significa que el valor no se modifica dentro del procedimiento), pero al final, los **cambios** hechos en el parámetro **se reflejan en la variable original**.

Por lo tanto, para simular el **pasaje por valor-resultado** en Pascal, deberíamos hacer lo siguiente:

1. Pasar x como valor al principio (para que no se modifique en la llamada a la función).

2. **Hacer que** Tres mantenga los cambios al final, utilizando el valor calculado en Dos para modificar x al final.

```
Program Uno;
var x: integer;

Function Dos: integer;
begin
    x := x + 1;
    return (x);
end;

Procedure Tres (valor_resultado x: integer);
begin
    x := x + 5;           // Paso por valor, x se modifica
    x := Dos + 10;        // x ahora se modifica, llamando a Dos
end;

begin
    x := 8;
    Tres(x);              // Llamamos a Tres
    write (x);            // Imprime el resultado final de x
end.
```

b)

Transcribimos el código a **Ada** y mostramos la diferencia entre **pasaje por referencia** y **pasaje por valor-resultado**.

1. Pasaje por Referencia en Ada:

En Ada, el pasaje por referencia se puede lograr usando el **tipo** `in out` en los parámetros. Esto significa que el valor de la variable se puede modificar directamente.

```
Program Uno is
    x : Integer := 8; -- Declaración de la variable global
    function Dos return Integer is
    begin
        x := x + 1; -- Incrementa x
        return x;   -- Retorna el nuevo valor de x
    end Dos;

    procedure Tres(x : in out Integer) is -- Pasaje por referencia
```

```

begin
  x := x + 5;           -- Modifica el valor de x
  x := Dos + 10;        -- Llama a Dos y modifica x
end Tres;

begin
  Tres(x);              -- Llama a Tres pasándole x por referencia
  Put_Line(Integer'Image(x)); -- Imprime el valor de x
end Uno;

```

```

Program Uno is
  x : Integer := 8; -- Declaración de la variable global
  function Dos return Integer is
  begin
    x := x + 1; -- Incrementa x
    return x;   -- Retorna el nuevo valor de x
  end Dos;

  procedure Tres(x : in out Integer) is -- Simulando el valor-resultado con
in out
  begin
    x := x + 5;           -- Modifica el valor de x
    x := Dos + 10;        -- Llama a Dos y modifica x
  end Tres;

begin
  x := 8;                -- Inicializa x
  Tres(x);               -- Llama a Tres pasando x por "valor-resultado"
  Put_Line(Integer'Image(x)); -- Imprime el valor de x
end Uno;

```

En Ada, el pasaje de parámetros se define mediante el uso de las palabras clave in, out y in out. La palabra clave in indica que el parámetro se pasa por valor, la palabra clave out indica que se pasa por referencia (es decir, se permite la modificación dentro del procedimiento o función), y la palabra clave in out indica que se pasa por valor-resultado.

Puntos debiles/ temas mas debiles que tengo en esta practica:

Pasaje por nombre, es como que usa una referencia de la variable pero de forma indirecta. (Ademas se invoca textualmente como se mando en el parametro real cada vez que se usa.)

pascal

 Copiar código

```
a := 5;
ejemplo3(a);      // por referencia: a se vuelve 6
ejemplo3(a + 1);  // por nombre: se transforma en: (a + 1) := (a + 1) + 1 →
```

⚠ Aquí es donde está la diferencia: con pasaje por nombre, si pasás una expresión como `a + 1`, se sustituye tal cual y no siempre se puede asignar a eso. En cambio, con pasaje por referencia, solo podés pasar una variable y la podés modificar.

Respuesta : El pasaje por nombre toma *textualmente* la expresión que le mandamos, como si la reemplazara en el código del procedimiento cada vez que se usa.

Esto quiere decir que si pasás `a + 1`, se evaluará `a + 1` cada vez que se use el parámetro dentro del procedimiento, no una sola vez.

Conclusión:

Entonces, por lo que tengo entendido, el **pasaje por nombre toma la expresión textualmente**, y si nosotros le pasamos `a + 1` por parámetro, **SIEMPRE** que se use ese parámetro dentro del procedimiento, se estará **evaluando nuevamente** `a + 1`.

📌 *Detalle importante:* no se hace una copia del valor, sino que se **sustituye la expresión original directamente en cada uso del parámetro formal**, lo que puede **provocar cambios dinámicos en tiempo de ejecución** si la expresión involucra variables que cambian.

Aunque **en algunos casos puede comportarse como pasaje por referencia**, no es lo mismo: el pasaje por nombre **no apunta a una ubicación fija en memoria**, sino que **re-evalúa la expresión original cada vez**.

Ejemplo 1: No modificamos el valor de 'a' dentro del procedure.

por eso cuando imprimimos a en el programa principal, el valor sigue siendo 1.

```
procedure mostrar (nombre a : integer)
begin
    write (a); // imprime 2
    write (a); // imprime 2
    // imprime 2 por que cada vez que se llame al parametro "a"
    // Se estaria ejecutando la expresion 'a+1'
end;
```



```
// programa principal
var
a : integer;
begin
    a := 1
    mostrar(a+1)
    write (a); // Imprime 1
end.
```

Ejemplo 2: Queremos modificar el valor de a dentro del procedure : Se genera un error!

```
procedure mostrar (nombre a : integer)
begin
    a := a + 1;
    // ✗ ERROR: intento de asignar a una expresión (a + 1)
    // Se genera un error por que se quiere hacer a+1:= a+1; ✗ ¡Esto es
    inválido!

end;
```

Ejemplo 3: Se modifica la variable dentro del procedimiento, el cambio se ve afectado fuera del procedimiento tambien.

Utiliza como una especie de pasaje de referencia de manera indirecta. Dentro del procedure,. trabaja sobre la variable y no trabaja sobre una copia.

```
procedure mostrar (nombre a : integer)
begin
    a := a+1;
    write (a); // imprime 2
end;

// programa principal
var
a : integer;
begin
    a := 1
    mostrar(a);
    write (a); // imprime 2
end;
```

Diferencia entre enlace estatico/dinamico y deep binding y shallow binding.

Conclusión clara:

Concepto	¿A qué aplica?	¿Qué decide?
Enlace estático	Variables	Usa el entorno donde la subrutina fue definida.
Enlace dinámico	Variables	Usa el entorno desde donde se invocó.
Deep binding	Subrutinas pasadas por parámetro	Amarra el entorno en el momento del pasaje.
Shallow binding	Subrutinas pasadas por parámetro	Amarra el entorno en el momento de ejecución.

 En resumen: *static/dynamic* = variables no locales; *deep/shallow* = subrutinas como parámetros.

Tipo de binding	¿Cuándo se enlaza D con un entorno?	¿Qué entorno usa para resolver x, y?	Resultado
Deep binding	En el momento en que D se pasa como parámetro (C(D))	El entorno léxico donde D fue definida (en Program A)	Usa x y y globales
Shallow binding	En el momento en que D se ejecuta (S(x, y))	El entorno dinámico de la llamada (el de C)	Usa x y y locales de C

Deep Binding:

- La subrutina **se amarra (se enlaza)** al **entorno donde fue definida**, cuando es pasada como parámetro.
- Entonces, cuando se la ejecuta después, **usa las variables no locales del entorno donde fue escrita**, *no de quien la llama*.
-  **No se amarra a donde se invoca**, sino a **donde fue definida**.

Shallow Binding:

- La subrutina se amarra **al entorno donde se ejecuta**, o sea, **quien la llama**.
- Las variables no locales se buscan en el **contexto dinámico** de ejecución.



Frase para Recordar:

- **Deep** → **Definición**
- **Shallow** → **Llamada**

La diferencia entre ambos está **en cómo se resuelven las variables no locales** de la función `D` (en este caso `x` `y` y `y`).

La técnica de binding (shallow o deep) **determina cómo se amarran las variables** cuando se ejecuta la función `D`.

6

- **Deep binding:** **`D` se amarra a las variables del entorno donde fue definida**. Esto significa que en el caso de tu código, `D` **usará las variables `x` `y` y `y` del programa principal** (donde se definió).
- **Shallow binding:** **`D` se amarra a las variables del entorno donde fue llamada**. Esto significa que `D` **usará las variables `x` `y` y `y` de `C`**, ya que `C` fue quien la llamó.